CodeQuest — Estado do projeto e histórico de versões

Documento vivo. Descreve tudo o que o CodeQuest tem hoje e registra, versão por versão, o que cada atualização acrescentou.

Versão atual	`v1.11.2`
Última atualização	01/09/2026
Migração mais recente	`0015_conquistas_no_perfil_publico.sql`
PDF	`CodeQuest-Versoes.pdf` — gerado por `npm run doc:pdf`

> As versões foram numeradas a partir do histórico do Git. Cada commit de funcionalidade virou uma versão *minor* (`1.5`, `1.6`...) e cada commit só de correção ou de acabamento virou uma *patch* (`1.4.1`, `1.8.1`...). Como manter isso daqui pra frente está na seção 6.

Sumário

1. O que é o CodeQuest
2. Os números de hoje
3. Tudo o que já existe
4. Histórico de versões
5. Mapa das migrações SQL
6. Como manter este documento

1. O que é o CodeQuest

Uma plataforma gamificada para aprender programação, no estilo Duolingo: o aluno escolhe uma linguagem, avança por módulos que só abrem em ordem, escreve código de verdade e recebe correção imediata, acumulando XP, níveis, streak, conquistas e posição no ranking.

Projeto Integrador de **Davi B. Tarquinio, Luiz A. S. P. Jr e Kauan F. Arruda**.

Três decisões atravessam o projeto inteiro:

- **Custo zero, sem cartão.** Nenhum serviço usado cobra, exige cartão ou vira pago depois de um teste. Nada é publicado na Google Play nem na Microsoft Store, que cobram taxa de desenvolvedor.
- **Uma base de código, quatro formatos.** O mesmo código vira site (Vercel), `.apk` (Capacitor), `.exe` e Linux (Tauri).
- **O servidor nunca confia no cliente.** XP, resultado de partida, elo e desafio diário são decididos no banco, por funções `SECURITY DEFINER`, com RLS por cima.

2. Os números de hoje

Linguagens no catálogo	**5** (Python e JavaScript com trilha completa)
Lições	**58** — Python 32, JavaScript 26
Exercícios	**116** — Python 62, JavaScript 54
Conquistas	**45**
Tags de perfil	**22**
Perguntas do duelo	**88** — JS 40, Python 12, Java 10, geral 10, C# 8, C++ 8
Telas	**14** rotas
Migrações SQL	**15**
Suítes de teste	**14**
Divisões do ranqueado	**22**, em 8 patamares

3. Tudo o que já existe

3.1 Trilhas e execução de código

- **Python, 9 módulos**: Fundamentos → Variáveis → Condicionais → Repetições → Listas → Dicionários → Funções → Desafios → Projeto Final.
- **JavaScript, 7 módulos**: Fundamentos → Variáveis → Condicionais → Repetições → Funções → Desafios → Projeto Final.
- Cada lição tem **teoria escrita**, exemplos e exercícios; o módulo N só abre com o N−1 inteiro concluído, e a rota `/licao/[id]` valida esse bloqueio (não adianta digitar a URL).
- Três tipos de exercício: **múltipla escolha**, **resposta digitada** e **código executado de verdade**.
- **Python roda no próprio aparelho** via Pyodide; **JavaScript roda num Web Worker** com o motor do próprio navegador. Nenhum serviço externo, nenhum custo por requisição, funciona offline.
- O worker de JavaScript tem `fetch`, `XMLHttpRequest`, `WebSocket`, `importScripts`, `indexedDB`, `caches` e `Worker` removidos antes de rodar o código do aluno; laço infinito morre por `terminate()`.
- **Validação estrutural**: além de conferir a saída, 77 exercícios exigem que a solução realmente use o que a lição ensinou — um `for` de verdade, não um `print(5050)` com a resposta na mão. E a regra de cada exercício só pode exigir nomes que o próprio enunciado dá, para nunca reprovar quem resolveu certo.
- As **alternativas são embaralhadas** por aluno, tanto na trilha quanto no duelo.
- As soluções de referência são executadas de verdade nos testes: **44/44 em Python, 33/33 em JavaScript**.

3.2 Gamificação

- **XP** de 20 a 120 por exercício; **nível = XP ÷ 100 + 1**; refazer lição não dá XP de novo.
- **12 títulos de nível**, de Iniciante (1) a Mito (50): Iniciante, Aprendiz, Programador, Engenheiro, Arquiteto, Veterano, Mestre, Grão-mestre, Lenda do código, Lenda viva, Fenômeno, Mito.
- **Streak** por data local: estudou ontem soma 1, pulou um dia volta pra 1.
- **45 conquistas**, avaliadas a cada conclusão, com cartão animado no desbloqueio.
- **22 tags** de perfil, incluindo quatro **secretas** e a tag exclusiva **Desenvolvedor/Dono**, atribuída só por campo direto no banco.
- **Ranking** por XP, com aba Global e aba Amigos.

3.3 Social

- **@username** único, com filtro de palavrões e troca liberada **só a cada 24 h** — bloqueado no banco, não só na tela.
- Pedidos de amizade: buscar, enviar, aceitar, recusar, remover.
- **Perfil público** de qualquer usuário em `/usuario`, mostrando XP, nível, streak, tags e conquistas — e **nunca o e-mail**. O progresso lição a lição também fica de fora: ele é privado.
- **Foto de perfil** com recorte e compressão no próprio aparelho.

3.4 Duelo 1v1 (PvP)

- Convite direto para um amigo, **5 perguntas** cronometradas, bônus por velocidade.

- Sincronização em tempo real pelo Supabase Realtime.
- **O placar é apurado no banco.** O cliente envia a resposta; quem decide o que é acerto, o bônus e quem venceu é a função no Postgres. Os testes incluem tentativas explícitas de trapaça.
- Seis tags de duelo, da estreia ao "Imortal", mais a conquista de **empate** — que vale nos dois modos, duelo e ranqueada.
- **Trocar de aba tem consequência**: o relógio não para, sumir mais de 5 s numa pergunta zera aquela pergunta, e a terceira vez encerra a partida como derrota.
- Aceitar um convite leva **os dois jogadores** para a partida ao mesmo tempo, de qualquer tela.
- **A navegação do aplicativo trava durante a partida**, e o cronômetro de cada pergunta é carimbado pelo servidor: sair e voltar retoma o tempo que sobrou, nunca os 20 s inteiros.

3.5 Modo ranqueado

- **8 patamares** — Bronze, Prata, Ouro, Diamante, Mítico, Lendário, Mestre (3 sub-níveis) e **Elite** (sem sub-níveis) — somando **22 divisões**, 100 de elo cada.
- Ganho e perda mudam conforme sobe: **+40/−10** no Bronze, **+13/−25** no Mestre, **+10/−30** no Elite. Lá em cima, manter posição custa tanto quanto chegar.
- **O patamar conquistado não se perde**: existe um piso por patamar, então derrota no piso não tira elo (e a tela explica isso, em vez de dizer "empate").
- **Temporadas de 15 dias**, que viram sozinhas — sem cron, sem serviço agendado: a primeira pessoa que abre a tela depois do fim dispara a virada.
- Perfil guarda o **pico histórico**, o patamar máximo alcançado e **em quantas temporadas** ele foi repetido.
- O elo é calculado **inteiramente no servidor**; o cliente nunca envia valor de elo.
- **Leaderboard em dois recortes**: a temporada atual e o hall da fama de todas as temporadas.

3.6 Desafio diário

- **Um exercício de código, o mesmo para todo mundo**, valendo **120 XP** — mais que qualquer exercício avulso, menos que uma lição inteira.
- A escolha é determinística: `(dias desde a época × 7919) mod tamanho-do-pool`. Como 7919 é primo e maior que o pool, **todos os exercícios saem antes de qualquer repetição**.
- A virada também é preguiçosa: não existe tarefa agendada, o dia é calculado na hora da consulta.
- "Já completei hoje?" é decidido **no banco** — a função de concluir não aceita nenhum parâmetro, nem data, nem usuário.
- Contador total no perfil, indicador no dashboard e **5 tags de "Diarista"**.

3.7 Contas e segurança

- Login por e-mail/senha ou **Google**, pelo Supabase Auth.
- **RLS em todas as tabelas.** As de conclusão de desafio não têm *nenhuma* política de escrita: só a função `SECURITY DEFINER` grava ali.
- **Modo demonstração**: sem as chaves do Supabase o app inteiro funciona pelo `localStorage` — ideal para desenvolver e para apresentar sem internet.

3.8 Quatro formatos, uma base de código

FORMATO	TECNOLOGIA	ARQUIVO
🌐 Site	Next.js na Vercel	—
Android	Capacitor	`codequest.apk`
Windows	Tauri	`.exe`
Linux	Tauri	`.AppImage` / `.deb`

O build estático (`NEXT_OUTPUT=export`) é o que alimenta o `.apk` e o `.exe`.

3.9 Qualidade

Quatorze suítes rodam num **Postgres de verdade** (PGlite), aplicando as migrações reais e testando com o RLS ligado:

`test:migrations` · `test:progresso` · `test:amigos` · `test:tags` · `test:pvp` · `test:ranqueado` · `test:desafios` · `test:estrutura` · `test:realtime` · `test:seo` · `test:conquistas` · `test:entrada`

Mais `content:check` (integridade do currículo), `content:verify-python` e `content:verify-js` (soluções de referência executadas), `typecheck`, `build` e `build:static`.

---

4. Histórico de versões

Da mais recente para a mais antiga.

---

v1.11.2 — 01/09/2026 · os títulos de nível vão até o 50

Novo — a escada de títulos parava no nível 20

Quem passava do nível 20 ficava com "Lenda do código" para sempre. Eram 6 degraus; agora são **12**, e o último é o **nível 50** — o mesmo teto da tag mais alta do jogo, "Lenda do CodeQuest".

NÍVEL	TÍTULO	NÍVEL	TÍTULO
1	Iniciante	25	Mestre
3	Aprendiz	30	Grão-mestre
6	Programador	35	Lenda do código
10	Engenheiro	40	Lenda viva
15	Arquiteto	45	Fenômeno
20	Veterano	50	Mito

- **"Lenda do código" subiu do nível 20 para o 35.** Ela é claramente um título de chegada, e ficava estranha a 40 % do caminho. Quem já estava acima do 35 **não perde nada** — continua com ela, e agora tem mais três degraus pela frente.
- Quem está entre o 20 e o 34 passa a ver Veterano, Mestre ou Grão-mestre. É a única mudança que troca um título por outro, e ela é honesta: nível 20 de 50 não é o topo.
- **"Desenvolvedor" (nível 10) virou "Engenheiro".** `Desenvolvedor` é o nome da tag exclusiva de quem construiu o CodeQuest, e ela pode estar equipada em qualquer nível — um aluno no nível 12 aparecia como "Nível 12 · Desenvolvedor", igualzinho ao selo do dono.

A escada virou uma tabela, e `content:check` a vigia

Estender agora é acrescentar uma linha em `TITULOS_DE_NIVEL`. O `content:check` recusa se:

- o primeiro degrau não for o nível 1, ou o último não for o nível 50;
- os níveis não subirem, ou houver título repetido;
- um título coincidir com o nome de uma **tag que possa aparecer no mesmo nível**. A conferência é nível a nível, e não por nome: a tag "Arquiteto" é do nível 20 e o título homônimo vai do 15 ao 19, então os dois nunca se encontram e a regra não reclama à toa.

SQL: nenhum — o título é calculado na tela, a partir do nível.

---

v1.11.1 — 01/09/2026 · as conquistas voltam a aparecer no perfil dos outros

Corrigido — o perfil público mostrava "0 de 45" para todo mundo

Abrir o perfil de outra pessoa mostrava a seção 🏆 Conquistas sempre vazia, com "Nenhuma conquista desbloqueada ainda" — inclusive para quem tinha dezenas delas. As tags, logo acima, apareciam normalmente.

- **A causa.** `user_tags` ganhou leitura pública na migração 0004, quando o perfil público foi criado. `user_achievements` ficou com a policy da 0001: `using (auth.uid() = user_id)` — só as suas. A tela sempre pediu as conquistas do outro, e o banco sempre devolveu zero linhas.
- **Por que passou tanto tempo despercebido.** RLS **filtra linhas, não levanta erro**. A consulta volta com sucesso e uma lista vazia, e lista vazia é indistinguível de "essa pessoa não tem conquista nenhuma". Não havia mensagem de erro em lugar nenhum — nem na tela, nem no console.
- E nenhum teste podia ver isso: todos rodavam como superusuário do Postgres, que **ignora RLS**. A suíte nova `test:perfil-publico` faz `set role authenticated` de propósito — é a única forma de a policy realmente valer. Ela reproduz o bug com as migrações até a 0014 e mostra ele sumir com a 0015.

O que a correção não abriu

- Escrita continua restrita ao dono: ninguém concede conquista, nem apaga a de outro (testado).
- **O progresso lição a lição continua privado** — `user_progress` não foi tocado.
- E-mail continua fora de alcance: ele nem existe em `profiles`, mora em `auth.users`.

SQL: `0015_conquistas_no_perfil_publico.sql`.

---

v1.11 — 01/09/2026 · Listas e Dicionários na trilha de Python

Dois módulos novos no meio da trilha, e a correção de um exercício que estava recusando a resposta certa.

Corrigido — o exercício do parêntese recusava quem o resolvia

`py-fund-04-e2` pede para consertar `print("Tudo certo!"` — falta fechar o parêntese. Escrever `print("Tudo certo!")`, a única resposta possível, era **recusado**, com a mensagem "Corrija o nome da variável no print()". Uma variável que aquele exercício nunca teve.

- **A causa.** A regra de estrutura tinha sido copiada do exercício de mesmo número da trilha de JavaScript. Lá o exercício é outro: um erro de digitação no nome de uma variável (`let mensagem` / `console.log(menssagem)`), e exigir `console.log(mensagem)` faz todo o sentido. Em Python o exercício é um parêntese sem fechar, e a regra exigia `print(mensagem)` — impossível de cumprir.
- **Por que nenhum teste pegou.** `test:estrutura` confere a regra contra a solução de referência, e a solução de referência tinha sido escrita para satisfazer a *regra* em vez de resolver o *exercício* (`mensagem = "Tudo certo!"` / `print(mensagem)`). Regra e solução concordavam uma com a outra e discordavam do enunciado. A verificação era circular.
- **O conserto do buraco, não só do caso.** `test:estrutura` ganhou uma checagem nova: todo nome próprio que uma regra **exige** tem de aparecer no próprio exercício — enunciado, código inicial, dica ou explicação. Uma regra que pede `mensagem` num exercício que nunca fala em `mensagem` foi copiada de outro lugar, e agora o teste diz isso.
- E o verificador de saídas em Python deixou de manter uma **segunda cópia** das soluções de referência: ele lê o mesmo `solucoes-referencia.json` que o outro teste. As duas cópias já tinham divergido — era a antiga que ainda guardava a solução errada.

Novo — Listas e Dicionários

A ordem da trilha de Python passou a ser: Fundamentos → Variáveis → Condicionais → Repetições → **Listas** → **Dicionários** → Funções → Desafios → Projeto Final.

- 📋 **Listas** — 4 lições: criar e acessar por índice (incluindo o índice negativo), `append` / `remove` / `pop` e alteração por posição, percorrer com `for ... in` e fatiar, e resumir com `sum`, `max`, `min`, `sorted` e `join`.
- 🗂️ **Dicionários** — 4 lições: chave e valor contra a posição da lista, consulta segura com `get()` / `in` / `del`, percurso com `items()` / `keys()` / `values()`, e o padrão de contagem *ler com padrão, somar, guardar de volta*.
- 8 lições, 16 exercícios, 2 medalhas de módulo. As 44 soluções de referência de Python foram executadas de verdade: **44/44**.
- A trilha inteira foi jogada no navegador, exercício por exercício: **62/62 aceitos**.

Novo — quem já tinha concluído a trilha

Como os módulos entraram no meio, quem terminou a trilha antes não está mais completo.

- A medalha **Trilha de Python concluída** e a tag **Pythonista** são devolvidas de quem ainda tem lições por fazer — e só elas.
- **XP, nível, streak, progresso das lições e todas as outras conquistas ficam intactos.** Quem fez Fundamentos até Funções fez de verdade.
- As duas voltam sozinhas ao concluir os módulos novos, pela avaliação normal ou por `conquistas_reavaliar()`.
- A migração **se recusa a rodar** se o `seed.sql` novo ainda não tiver sido aplicado — sem as lições novas no banco, ela puniria quem está em dia.
- Suíte nova `test:trilha` (23 verificações) cobre exatamente isso, num Postgres de verdade.

SQL: `seed.sql` (regerado) **e depois** `0014_trilha_python_listas_dicionarios.sql`.

---

v1.10.2 — 27/08/2026 · a barra do cronômetro presa ao relógio, e o empate 0 a 0 fora da medalha

Duas correções encontradas em uso.

A barra do cronômetro andava mais rápido que o relógio

A barra dava trancos a cada segundo e chegava ao fim com a pergunta ainda pela metade — o print mostrava a barra vazia com `9s` no contador.

- **A causa.** A barra é uma animação CSS de `transform` (a GPU cuida dela, sem JavaScript por quadro). Só que a v1.10 recalculava a **duração** e o **ponto de partida** dessa animação a cada redesenho da tela — ou seja, uma vez por segundo, quando o contador muda. O navegador *não* reinicia uma animação porque a duração mudou: ele mantém o tempo já corrido e passa a medi-lo contra a duração nova, sempre menor. A cada segundo a barra pulava para a frente e acelerava; aos 10 s ela já estava zerada, com metade da pergunta por responder.

- **A correção.** A animação passou a ser sempre a mesma: de cheia a vazia, na duração inteira da pergunta. Quem entra numa pergunta já em andamento não muda a animação — entra nela adiantado, com `animation-delay` negativo. Os dois valores são gravados uma única vez, quando a pergunta abre, e nenhum redesenho os toca.

Testado lendo a matriz de transformação real da barra na tela, segundo a segundo, contra o contador: o desvio máximo ficou em 0 %, a queda por segundo em 5,1 % (o esperado para 20 s) e a barra nunca zerou com o contador andando. Sair da tela e voltar continua devolvendo a barra no ponto certo, não cheia.

Empate em 0 a 0 não vale mais a medalha de empate

"Empate técnico" foi criada para ser a medalha mais rara do jogo: exige placar idêntico até o bônus de velocidade, contado em milésimos de segundo. Só que havia um empate fácil — o 0 a 0. Contra o bot da ranqueada, bastava não responder nada por dois minutos. A medalha mais rara era, na prática, a mais farmável.

- Empate só conta para a medalha se **alguém pontuou**.
- A conta passou a sair do **histórico de partidas**, e não dos contadores. `pvp_stats.empates` e `ranked_stats.empates` somam quantos empates houve, nunca com que placar — não há como descontar o 0 a 0 de um número já somado. `matches` guarda o placar de cada partida, então a regra nova vale retroativo sem migração de dados.
- Os contadores do perfil **não mudaram**: 0 a 0 continua aparecendo como empate no cartel, porque é um empate. Só não é um feito.
- Quem tirou a medalha pela regra antiga a **devolve** — a migração faz essa limpeza uma vez, e só para este critério (é o único cujo histórico completo está no banco; o *streak*, por exemplo, só guarda o valor corrente, e reavaliar tirando medalha puniria quem apenas deixou a sequência morrer).

SQL: `0013_empate_zero_a_zero.sql`.

---

v1.10.1 — 26/08/2026 · o cronômetro para de depender do relógio do aparelho

Conserto de uma regressão da v1.10, encontrada em uso: no computador, a partida abria já com o tempo esgotado — a tela pulava sozinha para "Errou", marcava `0s` e a partida inteira passava com 0 ponto. No celular não acontecia.

Corrigido

- **A conta misturava dois relógios.** A v1.10 calculava `20s − (Date.now() do navegador − começou_em do servidor)`. Enquanto os dois relógios concordam, o resultado está certo; num PC com a hora adiantada em mais de 20 s — coisa comum, e que o celular não tem porque acerta a hora pela rede — a pergunta já nascia vencida.
- `pvp_pergunta` passou a devolver também o `agora` **do servidor**. Com os dois valores saindo do mesmo relógio, o tempo decorrido é exato e a hora do aparelho deixa de entrar na conta. De quebra, um erro de fuso ou de formato na leitura da data também some: afetaria os dois carimbos igualmente e se cancela na subtração.
- Sem a migração aplicada, a tela agora **descarta** um tempo decorrido impossível (negativo ou maior que a própria pergunta) e dá o tempo cheio, em vez de zerar a pergunta na cara de quem está jogando.

Testado com o relógio do navegador deslocado de propósito — adiantado 1 minuto, adiantado 6 horas, atrasado 1,5 minuto e certo (controle): nos quatro, a pergunta abre com tempo e aceita resposta.

SQL: `0012_relogio_do_servidor.sql`.

---

v1.10 — 26/08/2026 · versão na tela, partida sem pausa e fluidez no celular

Adicionado

- **Selo de versão** no canto inferior esquerdo, em todas as telas. A fonte é `src/lib/versao.ts`, e `npm run content:check` **recusa** se ela divergir da versão deste documento — assim o site e o histórico não contam versões diferentes.
- **A navegação trava durante a partida.** Trocar para Início, Amigos ou Trilha no meio do duelo desmontava a tela e virava uma pausa de graça. Agora os links ficam inertes até a partida acabar, com a razão escrita na tela, e fechar ou recarregar pede confirmação ao navegador.
- **O cronômetro é do servidor.** `pvp_pergunta` passou a devolver o carimbo da primeira entrega, então voltar para uma pergunta já começada retoma o tempo que sobrou.

Corrigido

- **O relógio voltava cheio** ao sair da tela e voltar. O placar nunca foi enganado — o servidor sempre contou desde a primeira entrega — mas a tela mostrava 20 s de novo, e a pessoa respondia com calma e levava 0 sem entender.
- **Pontuação da partida anterior aparecia na seguinte.** Ir para outra partida nem sempre desmonta a tela (a rota é a mesma, muda só o `?m=`), e o estado sobrevivia. Agora tudo é zerado quando a partida muda.
- **"Você saiu da aba 3 vezes" aparecia em partida ganha.** A mensagem saía de um contador do navegador, que herdava o valor da partida anterior. Agora ela vem de `matches.abandonada_por`, no banco: só aparece quando o servidor registrou o abandono.
- **Abandono que falhava mostrava um resultado inventado.** Se `pvp_abandonar` não respondia (a migração 0010 ainda não aplicada, por exemplo), a tela ia para o fim mesmo assim, com placar parcial. Agora só encerra se o servidor encerrou.
- **A 0011 quase apagou o embaralhamento das alternativas** — a função foi reescrita a partir da versão errada. `npm run test:pvp` pegou: a resposta certa voltou a cair na primeira posição, 40 vezes em 40.

Fluidez no celular

- O brilho de fundo saiu do `background-attachment: fixed` para uma **camada fixa própria**. Fundo preso obriga o navegador a repintar os três gradientes a cada quadro de rolagem, no processador — era o que impedia a rolagem de acompanhar uma tela de 120 Hz.
- `backdrop-blur` **só no desktop**: cabeçalho, barra inferior e cards. Desfocar o fundo é a conta mais cara da interface no celular, e o fundo dos cards já é opaco o bastante.
- O cronômetro do duelo passou de **10 renderizações por segundo para 1**; o movimento contínuo da barra virou animação CSS de `transform`, que a GPU resolve sozinha.
- `touch-action: manipulation` nos alvos de toque, para o navegador não segurar o clique esperando um duplo-toque.

Medido com a mesma página, o mesmo aparelho emulado e a mesma rolagem — só o CSS muda, com o processador afunilado em 4× (três execuções):

	QUADROS PERDIDOS	PIOR QUADRO
CSS antigo	5, 42 e 19 (de ~120)	33 a 50 ms
CSS novo	**0, 0 e 0**	**16,8 ms**

SQL: `0011_cronometro_e_abandono.sql`.

---

v1.9 — 26/08/2026 · empates, hall da fama e punição por trocar de aba

Adicionado

- **Empates no cartão de Ranqueada.** Vitórias / Derrotas / Empates em destaque, no mesmo padrão do cartão de Duelos; o total de partidas desceu para a linha de texto, com o aproveitamento.
- **A conquista de empate passou a valer na ranqueada.** Ela lia só `pvp_stats`, e o resultado da ranqueada vai para `ranked_stats` — quem empatou na ranqueada nunca recebia a medalha. Agora a contagem soma os dois modos. As **vitórias** continuam separadas de propósito: vitória contra bot no cartel 1v1 transformaria a tag "Imortal" numa tarde de farm.
- **Reavaliação retroativa de conquistas** (`conquistas_reavaliar`). Confere cada critério contra o histórico real no banco e concede o que faltou — conserta quem cumpriu o requisito antes de o critério existir. Roda sozinha ao abrir o perfil e ao terminar uma partida ranqueada.
- **Leaderboard "Todos os tempos"** na ranqueada, ao lado de "Temporada atual": ordena pelo maior pico de elo de todas as temporadas e não zera a cada 15 dias.
- **Saudação de conta nova.** Quem entra pela primeira vez vê "Bem-vindo, [nome]!"; depois disso, "Bem-vindo de volta". A primeira visita fica carimbada em `profiles.boas_vindas_em`.
- **Trocar de aba durante a partida agora custa.** O relógio nunca parou (ele é calculado por tempo de parede); o que faltava era consequência: sumir mais de **5 s** numa pergunta faz aquela pergunta valer 0, e na **3ª vez** a partida encerra como derrota, com a perda de elo normal. Um aviso na tela explica o que aconteceu e o que acontece se repetir.
- **Os dois jogadores entram na partida ao mesmo tempo**, de qualquer tela. Antes só quem aceitava o convite era levado; quem convidava ficava esperando enquanto o relógio já corria.
- Suítes novas: `test:conquistas` (30 testes) e `test:entrada` (37 testes).

Corrigido

- A saudação de conta nova era consumida pela tela de escolha de @username, que monta o painel por um instante antes de assumir a tela — o primeiro painel de verdade já dizia "de volta".

SQL: `0010_empates_leaderboard_abandono.sql`.

---

v1.8.2 — 26/08/2026 · documento de versões e PDF

Este documento passou a existir, e a virar PDF sozinho.

Adicionado

- `HISTORICO.md` — o resumo do que o projeto tem hoje e o histórico de todas as versões, com o SQL necessário em cada uma.
- `npm run doc:pdf` — gera o `CodeQuest-Versoes.pdf` a partir **deste mesmo arquivo**, usando o Chrome ou o Edge que já existe na máquina. Nenhuma dependência nova, nada que cobre, e um texto só para manter: o PDF nunca é escrito à mão.
- O PDF sai em tema claro, com quebra de página por seção — é feito para anexar em e-mail e para imprimir, não para ler no escuro.

Corrigido

- O README dizia "16 medalhas", número parado desde a v1.2. São 43.

SQL: nenhum.

---

v1.8.1 — 26/08/2026 · `217beb3` · correção de mais bugs

Três problemas encontrados em uso real, um deles causado pela própria v1.8.

Corrigido

- **A aba Amigos quebrava com "Application error".** Duas regras do Realtime se combinavam mal: `supabase.channel(tópico)` devolve o canal **já existente** se o tópico se repete, e registrar um ouvinte depois do `subscribe()` lança exceção. Como o aviso de convite fica montado em todas as telas, a aba Amigos pegava aquele canal e derrubava a página. Agora cada assinante monta um tópico próprio.
- **Derrota mostrava "Empate não mexe no elo".** A tela deduzia empate de "elo não mudou", mas isso também acontece ao perder no piso do patamar. Agora a mensagem explica o piso.
- **Pergunta do duelo com três respostas certas.** As alternativas eram `Ana`, `  Ana  `, `ana` e `"Ana"` — e o servidor normaliza antes de comparar, então três davam ponto.

Adicionado

- Conquista 🤝 **Empate técnico** (o resultado mais raro do jogo).
- `content:check` passou a comparar as alternativas **já normalizadas**, com o mesmo normalizador da correção — e achou um terceiro caso na trilha (`js-cond-04-e3`).
- Suíte `test:realtime`, que reproduz a exceção com a biblioteca real, sem rede.

SQL: nenhuma migração nova. Rodar o `seed.sql` de novo (conquista nova + perguntas corrigidas).

---

v1.8 — 26/08/2026 · `746199d` · patamar Elite e correções

Adicionado

- **Patamar Elite**, acima do Mestre, a partir de 2100 de elo. O Mestre voltou a ter 3 sub-níveis: 22 divisões no total.
- **Validação estrutural dos exercícios** (`src/lib/exercises/estrutura.ts` + 61 regras): não basta a saída bater, a solução precisa usar o que a lição ensinou.
- Contador de rank detalhado no perfil: patamar máximo, pico histórico e em quantas temporadas ele se repetiu — **contando** as temporadas, em vez de guardar um número que poderia dessincronizar.

Corrigido

- Convite de PvP aceito passou a levar direto para a partida.
- A trilha voltou a lembrar qual linguagem estava selecionada.

SQL: `0009_ranqueado_elite.sql` + `seed.sql`.

---

v1.7 — 25/08/2026 · `4835120` · polimento visual

Passe de acabamento **sem tocar em nenhuma lógica, rota, banco ou paleta**: estendeu o design system existente em vez de trazer uma biblioteca de componentes.

Adicionado

- Hierarquia visual em 3 níveis, tipografia e espaçamentos revisados.
- Componente `Field` unificando os formulários.
- Transições padronizadas de uma vez, pelo `tailwind.config.ts`.

SQL: nenhum.

---

v1.6 — 25/08/2026 · `3030757` · desafios diários

Adicionado

- Desafio do dia, igual para todos, valendo 120 XP, com escolha determinística e virada sem cron.
- Tabelas `daily_challenges` e `daily_challenge_completions`, com RLS **sem nenhuma política de escrita**.
- Contador no perfil, indicador no dashboard, 5 tags de "Diarista" e conquistas por total de desafios.
- Suíte `test:desafios`, que compara dia a dia a regra do banco com a do modo demonstração.

SQL: `0008_desafios_diarios.sql` + `seed.sql`.

---

v1.5 — 25/08/2026 · `b028c19` · trilha de JavaScript

Adicionado

- Trilha completa de JavaScript: **7 módulos, 26 lições, 54 exercícios**, com teoria escrita do zero.
- Motor de execução de JavaScript no navegador (`public/js-worker.js`), com o runtime (`public/js-runtime.js`) **compartilhado entre o navegador e o verificador em Node** — as duas execuções não podem divergir.
- `src/lib/runtime/executor.ts`, gerente genérico de worker usado por Python e JavaScript.
- Banco de perguntas de JavaScript do duelo ampliado de 12 para 40.
- `content:verify-js`, que roda as 33 soluções de referência de verdade.

SQL: `seed.sql`.

---

v1.4.1 — 24/08/2026 · `67a6611` · correções do ranqueado e do PvP

Adicionado

- **Embaralhamento das alternativas** por aluno, na trilha e no duelo.
- Escolha de tema no ranqueado e melhorias na tela de ranking.
- Suíte `test:pvp`, incluindo tentativas explícitas de trapaça.

SQL: `0007_embaralhar_alternativas.sql` + reaplicação de `0005` / `0006`.

---

v1.4 — 24/08/2026 · `bd88459` + `c4397ce` · modo ranqueado

Adicionado

- Modo ranqueado com elo, patamares, sub-níveis e **temporadas de 15 dias que viram sozinhas**.
- Bots por patamar para quando não há oponente humano.
- Todo o cálculo de elo no servidor; leaderboard por temporada.
- Suíte `test:ranqueado`.

SQL: `0006_ranqueado.sql` + `seed.sql`.

---

v1.3.1 — 24/08/2026 · `9b36a02` + `49a9981` · SEO

Adicionado

- Metadados por rota, `sitemap.xml`, `robots.txt` e imagem de compartilhamento (Open Graph) gerada por script.
- Verificação do Google Search Console.
- Suíte `test:seo`.

SQL: nenhum.

---

v1.3 — 23/08/2026 · `76b6b5c` · duelo 1v1 (PvP)

Adicionado

- Duelo de 5 perguntas cronometradas contra um amigo, em tempo real.
- Apuração do placar **no banco**, com bônus de velocidade.
- Banco inicial de perguntas do duelo e as tags de PvP.

SQL: `0005_pvp.sql` + `seed.sql`.

---

v1.2 — 23/08/2026 · `cde5626` · tags, perfis públicos e foto

Adicionado

- Sistema de **tags** de perfil, com tags secretas e a tag exclusiva Desenvolvedor/Dono.
- **Perfil público** em `/usuario`, sem expor e-mail.

- **Foto de perfil** com recorte e compressão local.
- Filtro de palavrões no @username.
- Suíte `test:tags`.

SQL: `0004_perfis_publicos.sql` + `seed.sql`.

---

v1.1.2 — 23/08/2026 · `20f55ea` · correções de progresso e username

Corrigido

- Perda e sobrescrita de progresso ao trocar de aparelho ou reconectar.
- Regras do @username endurecidas no banco (unicidade e troca a cada 24 h).

Adicionado

- Suítes `test:progresso` e `test:migrations`, rodando num Postgres real com RLS ligado.

SQL: `0003_protecao_progresso.sql`.

---

v1.1.1 — 22/08/2026 · `5129d65` + `1753572` · responsividade

Corrigido

- Layout em telas pequenas, logo e rodapé.
- Estouros de layout com a fonte do sistema aumentada (acessibilidade).

SQL: nenhum.

---

v1.1 — 22/08/2026 · `f96a7bb` · sistema de amigos

Adicionado

- **@username** único e obrigatório, com tela de escolha no primeiro acesso.
- Pedidos de amizade: buscar, enviar, aceitar, recusar, remover.
- Aba **Amigos** no ranking.
- Suíte `test:amigos`.

SQL: `0002_amigos.sql`.

---

v1.0 — 21/08/2026 · `fec0f67` · CodeQuest 1.0

A base inteira, de uma vez.

Adicionado

- Trilha de **Python** com 7 módulos, 24 lições e 46 exercícios.
- Execução de Python no navegador via Pyodide.
- XP, níveis, streak, conquistas, progressão bloqueada e ranking.
- Login por e-mail/senha e Google; RLS desde o primeiro dia.

- **Modo demonstração** completo, sem Supabase.
- Empacotamento para Android (Capacitor) e Windows/Linux (Tauri).

SQL: `0001_init.sql` + `seed.sql`.

---

5. Mapa das migrações SQL

Rodar em ordem, uma vez cada. O `seed.sql` é regerado por `npm run seed:gen` e pode ser rodado quantas vezes for preciso (ele é idempotente).

ARQUIVO	VERSÃO	O QUE TRAZ
`0001_init.sql`	v1.0	Perfis, progresso, currículo, conquistas, RLS
`0002_amigos.sql`	v1.1	@username e amizades
`0003_protecao_progresso.sql`	v1.1.2	Progresso à prova de sobrescrita; regra das 24 h
`0004_perfis_publicos.sql`	v1.2	Tags, perfil público, avatar
`0005_pvp.sql`	v1.3	Duelo 1v1 e apuração no servidor
`0006_ranqueado.sql`	v1.4	Elo, patamares e temporadas
`0007_embaralhar_alternativas.sql`	v1.4.1	Ordem das alternativas por aluno
`0008_desafios_diarios.sql`	v1.6	Desafio do dia
`0009_ranqueado_elite.sql`	v1.8	Patamar Elite e Mestre com 3 sub-níveis
`0010_empates_leaderboard_abandono.sql`	v1.9	Empates nos dois modos, reavaliação de conquistas, leaderboard histórico, abandono, saudação
`0011_cronometro_e_abandono.sql`	v1.10	Cronômetro carimbado pelo servidor e abandono no estado da partida
`0012_relogio_do_servidor.sql`	v1.10.1	A pergunta devolve também o "agora" do servidor
`0013_empate_zero_a_zero.sql`	v1.10.2	Empate 0 a 0 deixa de valer a medalha de empate
`0014_trilha_python_listas_dicionarios.sql`	v1.11	Devolve a medalha/tag de trilha completa de quem ficou com dois módulos por fazer
`0015_conquistas_no_perfil_publico.sql`	v1.11.1	Leitura pública de `user_achievements`, para as conquistas aparecerem no perfil dos outros

6. Como manter este documento

A cada atualização do CodeQuest:

1. **Escolha o número.** Funcionalidade nova → sobe a *minor* (`v1.8` → `v1.9`). Só correção ou acabamento → sobe a *patch* (`v1.9` → `v1.9.1`).
2. **Adicione a entrada no topo da seção 4**, seguindo o formato: título com versão, data, commit e assunto; um parágrafo de resumo; listas **Adicionado** e **Corrigido**; e a linha **SQL:** dizendo o que precisa rodar no Supabase (ou "nenhum").
3. **Atualize a tabela do topo** (versão atual, data, commit, migração).
4. Se os números mudaram, atualize a **seção 2** — `npm run content:check` imprime lições, exercícios, conquistas e perguntas do duelo já contados.
5. Se a atualização criou uma migração, acrescente a linha no **mapa de SQL**.
6. Se a atualização mudou o que o app faz, ajuste a **seção 3** — ela descreve o estado atual, não o histórico.
7. **Rode** `npm run doc:pdf`. Ele regenera o `CodeQuest-Versoes.pdf` a partir deste arquivo — é o que se anexa em e-mail. O PDF nunca é editado à mão: qualquer coisa escrita direto nele se perde na próxima geração.

> **Precisa do Chrome ou do Edge instalado** — é ele quem imprime. Em outra máquina, ou com o navegador em lugar diferente do usual: `CHROME_PATH="/caminho/do/chrome" npm run doc:pdf`.